

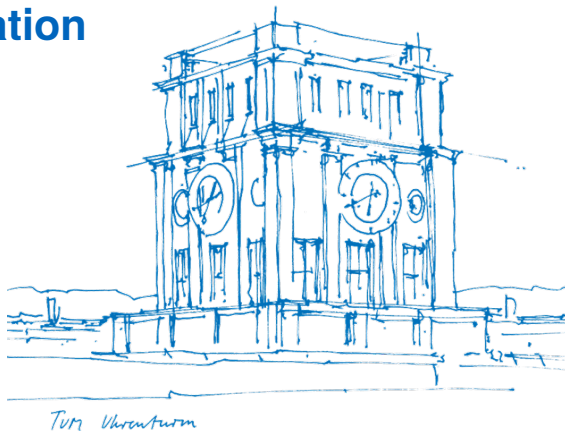
Agentic AI for Code Generation

Seminar Report — SS26

Thua Duc Nguyen

Supervisor: Derui Zhu
TUM School of Computation, Information and
Technology, Technical University of Munich

July 14, 2026



1 What works

2 What the numbers hide

What SWE-bench measures

One instance [13] = a real merged pull request that

1. closes a linked GitHub issue, *and*
2. modifies the test suite.

The agent sees the repository *before* the merge, plus the issue text. It returns a patch.

The patch is graded by running tests:

- *fail-to-pass* must flip,
- *pass-to-pass* must not regress.

Resolve rate = both hold, on the first patch (pass@1).

Split	Size	Kept only
Full	2,294	every scraped PR that touches tests
Lite	300	single-file patches; some issues leak the fix
Verified	500	human-screened: clear issues, fair tests
Pro	1,865	multi-file, long-horizon; held-out repos

Rates are not comparable across splits. Verified is the most *curated* split, not the hardest.

No scaffold wins twice

	SWE-agent custom ACI	OpenHands code-as-action	mini-SWE-agent ~100 lines of Python	spread
GPT-5-mini	38.8	38.6	51.8	13.2
Qwen3 Coder 480B	36.6	54.0	40.2	17.4
DeepSeek R1	22.6	18.0	11.0	11.6

Resolve rate (pass@1) on SWE-bench Verified, April 2026 [23, 24].

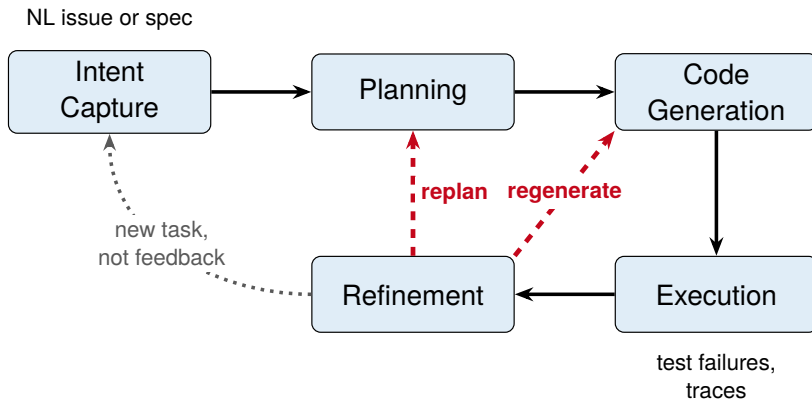
No scaffold wins twice

	SWE-agent custom ACI	OpenHands code-as-action	mini-SWE-agent ~100 lines of Python	spread
GPT-5-mini	38.8	38.6	51.8	13.2
Qwen3 Coder 480B	36.6	54.0	40.2	17.4
DeepSeek R1	22.6	18.0	11.0	11.6

Resolve rate (pass@1) on SWE-bench Verified, April 2026 [23, 24].

- Same model, same benchmark: the scaffold alone is worth **11–17 points**.
- **No scaffold wins twice.** The best scaffold *depends on the model* — and for GPT-5-mini the winner is the 100-line one.

The agent pipeline



Planning is localisation, not design

- **Where it sits** — input: an issue. Output: *a place to edit*.
- **What a “plan” is** — a *narrowing*: repository → files → classes/functions → edit locations.
- **What shape it takes** — a chain, a backtracking tree, or a merging graph.
- **What it contributes** — **grounding**, not deliberation.

Planning is not reasoning about *how* to write the code.
It is reasoning about *where* the code is.

Execution: the code meets the environment

- **Where it sits** — input: a patch. Output: *evidence*.
- **What actually runs** — the repository, checked out at the parent commit, inside a container [26]. The agent runs the test suite, the interpreter, the linter.
- **What comes back** — failing tests, tracebacks, a rejected edit. Not a score: *a symptom*, which still has to be read.
- **Why it is the expensive stage** — every observation costs a container round-trip, and the transcript grows with each one.

The only stage whose output the model did not write.

Everything in Act II turns on that sentence.

Refinement: what to do with the signal

Where it sits — input: whatever execution returned. Output: a *decision* — repair the patch, re-plan, or stop.

- **Repair from the trace** [12] — the traceback goes straight back into the prompt.
- **Explain your own code** [3] — discrepancies surface. Up to +12 %, but only where tests exist.
- **Sample N , rerank with a trained critic** [21] — OpenHands on Verified: 60.6 % → 66.4 %.

Agents improve their own code.

...or so this reads.

A map of the design space, not a ranking

System	Architecture	Backbone	Lite	Full
			<i>pass@1</i>	<i>pass@1</i>
SWE-agent [29]	OTA loop + ACI	GPT-4 Turbo	18.0 %	12.5 %
Agentless [27]	<i>Fixed pipeline</i>	GPT-4o	32.0 %	— [†]
AutoCodeRover [31]	OTA loop + AST search	GPT-4	19.0 %	12.4 %
OpenHands [26]	OTA loop + code-as-action	Sonnet 3.5	26.0 %	— [†]
Devin [4]	OTA loop + Brain/DevBox	undisclosed	—	13.9 % [‡]

OTA = observe–think–act loop; ACI = agent–computer interface (both defined next slide). [†]Never run on Full — the authors cite compute cost. [‡]Self-reported on a random 25 % subset.

Interface and context move scores as much as the model

1. **They all run an observe–think–act loop** [30] — observe, reason, act via a tool call.
2. **They all manage context explicitly** — no repository fits in the window, so each must choose what to read: repo skeleton + embeddings (Agentless), coarse-to-fine AST search (AutoCodeRover), observation compression (SWE-agent).
3. **They all build a purpose-built action interface** — none hands the model a raw shell. The agent only ever sees what its commands print back. Two opposite bets [29]:

	SWE-agent	OpenHands
The bet	Constrain the action space	Give it a general one
Reading	100-line file window	Python + bash,
Editing	Linters may <i>reject</i> the edit	in a Docker sandbox
Ablated	Remove one part of SWE-agent's interface: −3 to −6 pp (Lite)	

Interface and context strategy move resolve rates as much as the backbone does.

1 What works

2 What the numbers hide

The question this talk asks



**At each stage,
what can tell the model it is wrong?**

**At each stage,
what can tell the model it is wrong?**

Read as a control system: intent is the uncontrolled input, generation is the plant, execution is the oracle, planning and refinement are the control layer.

What can tell the model it is wrong? Three answers

Authoritative oracle

Test runner, compiler, human.
Ground truth.

It can tell the model something it
did not already believe — it can
surprise it.

What can tell the model it is wrong? Three answers

Authoritative oracle

Test runner, compiler, human.
Ground truth.

It can tell the model something it did not already believe — it can **surprise** it.

Learned / generated proxy

Trained critic, generated tests, LLM judge.

Carries real information — but never more than it was fitted with.
A generated test *runs* for real; the model still wrote the answer key.

Ceiling: its own fidelity.

What can tell the model it is wrong? Three answers

Authoritative oracle

Test runner, compiler, human.
Ground truth.

It can tell the model something it did not already believe — it can **surprise** it.

Learned / generated proxy

Trained critic, generated tests, LLM judge.

Carries real information — but never more than it was fitted with. A generated test *runs* for real; the model still wrote the answer key.

Ceiling: its own fidelity.

Same-distribution self-critique

The model judging itself.

No information from outside the model — the critic is drawn from the distribution that produced the error [11].

What can tell the model it is wrong? Three answers

Authoritative oracle

Test runner, compiler, human.

Ground truth.

It can tell the model something it did not already believe — it can **surprise** it.

Learned / generated proxy

Trained critic, generated tests, LLM judge.

Carries real information — but never more than it was fitted with. A generated test *runs* for real; the model still wrote the answer key.

Ceiling: its own fidelity.

Same-distribution self-critique

The model judging itself.

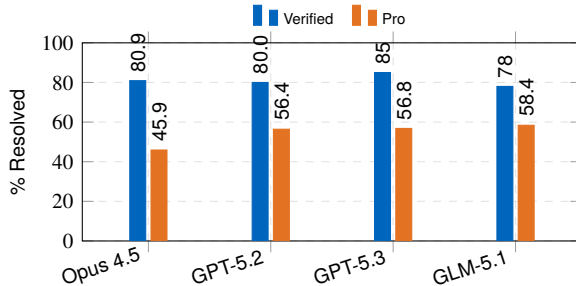
No information from outside the model — the critic is drawn from the distribution that produced the error [11].

An agent that could tell its wrong code from its right code by inspection would have written the right code in the first place.

It can reorganise what the model believes. It cannot add to it.

The benchmark can be gamed

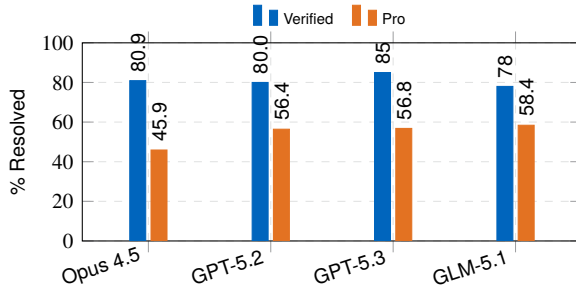
The grading tests *are* the oracle. **A weak oracle makes optimising against it *overfitting*.**



The *same weights* drop ≥ 20 points on the contamination-resistant SWE-bench Pro [6].

The benchmark can be gamed

The grading tests *are* the oracle. **A weak oracle makes optimising against it *overfitting*.**

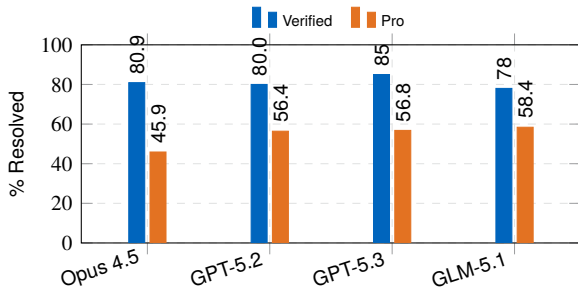


The *same weights* drop ≥ 20 points on the contamination-resistant SWE-bench Pro [6].

Read only the floor. Pro is harder *as well as* fresher, and the column mixes harnesses.

The benchmark can be gamed

The grading tests *are* the oracle. **A weak oracle makes optimising against it *overfitting*.**



The *same weights* drop ≥ 20 points on the contamination-resistant SWE-bench Pro [6].

Read only the floor. Pro is harder *as well as* fresher, and the column mixes harnesses.

- **91 %** of Verified is work a developer finishes *within the hour* [8].
- OpenAI dropped Verified in early 2026: models reproduce gold patches *from the task ID alone* [20].
- “Reward hacking is swamping model intelligence gains” [5].

Where a wrong guess is never contradicted

Intent capture

OpenAI's screen of 1,699 real issues **flagged 38.3 %** as *underspecified* [19].

No surveyed system asks a clarifying question. Each commits to a reading. **Open loop.**

The mechanism exists — TiCoder, ClarifyGPT, and an underspecified SWE-bench where the agent that *asks* wins [14, 17, 25].

Test-based validation

The same screen **flagged 61.1 %** for tests that *may unfairly mark a valid solution incorrect* [19].

That is the failure we can *see*. The one we cannot: a wrong guess about an *untested* invariant is never contradicted — it scores as success.

And augmenting the **proxy** inherits the generator's own misreading, exactly where the new tests were needed.

Where a wrong guess is never contradicted

Intent capture

OpenAI's screen of 1,699 real issues **flagged 38.3 %** as *underspecified* [19].

No surveyed system asks a clarifying question. Each commits to a reading. **Open loop.**

The mechanism exists — TiCoder, ClarifyGPT, and an underspecified SWE-bench where the agent that *asks* wins [14, 17, 25].

Test-based validation

The same screen **flagged 61.1 %** for tests that *may unfairly mark a valid solution incorrect* [19].

That is the failure we can *see*. The one we cannot: a wrong guess about an *untested* invariant is never contradicted — it scores as success.

And augmenting the **proxy** inherits the generator's own misreading, exactly where the new tests were needed.

Both benchmarks **hand-solve these two stages** before the agent starts: Verified *discards 68.3 %* of what it screened [19]; Pro pays experts to *rewrite* every issue [6].

No agent does either. Nothing ever asked it to.

Conclusion — three things the numbers showed

1. **The wrapper carries the score — but no wrapper carries it twice.**

The scaffold alone moved **11–17 points**, and the winning column changed row by row. What the wrapper *feeds* the generator — interface, context, localisation — does the work.

Conclusion — three things the numbers showed

1. The wrapper carries the score — but no wrapper carries it twice.

The scaffold alone moved **11–17 points**, and the winning column changed row by row. What the wrapper *feeds* the generator — interface, context, localisation — does the work.

2. Refinement pays exactly as much as its signal can contradict it.

Execution grounds the gains. A **trained critic** is a real gain ($60.6 \rightarrow 66.4\%$), but it is still a model — it cannot *surprise* the generator. **Self-critique** alone: no reliable gain, and it can degrade [11].

Conclusion — three things the numbers showed

1. The wrapper carries the score — but no wrapper carries it twice.

The scaffold alone moved **11–17 points**, and the winning column changed row by row. What the wrapper *feeds* the generator — interface, context, localisation — does the work.

2. Refinement pays exactly as much as its signal can contradict it.

Execution grounds the gains. A **trained critic** is a real gain (60.6 → 66.4 %), but it is still a model — it cannot *surprise* the generator. **Self-critique** alone: no reliable gain, and it can degrade [11].

3. The two stages nothing contradicts are where the numbers are softest.

Intent and **validation** have no gate — OpenAI's own screen flagged **38.3 %** and **61.1 %** of real issues — and the benchmark hid it: the same weights drop ≥ 20 points on Pro. Yet the one paper that *instrumented* intent lifted *all nine* cells of the grid on slide 3 [23] — its own control condition. The stage nobody instruments still pays out.

Conclusion — three things the numbers showed

1. The wrapper carries the score — but no wrapper carries it twice.

The scaffold alone moved **11–17 points**, and the winning column changed row by row. What the wrapper *feeds* the generator — interface, context, localisation — does the work.

2. Refinement pays exactly as much as its signal can contradict it.

Execution grounds the gains. A **trained critic** is a real gain (60.6 → 66.4 %), but it is still a model — it cannot *surprise* the generator. **Self-critique** alone: no reliable gain, and it can degrade [11].

3. The two stages nothing contradicts are where the numbers are softest.

Intent and **validation** have no gate — OpenAI's own screen flagged **38.3 %** and **61.1 %** of real issues — and the benchmark hid it: the same weights drop ≥ 20 points on Pro. Yet the one paper that *instrumented* intent lifted *all nine* cells of the grid on slide 3 [23] — its own control condition. The stage nobody instruments still pays out.

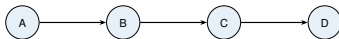
Impressive → dependable turns on letting **the environment say no** — because on this evidence, **the model will not say it to itself.**

Backup: every refinement mechanism, sorted

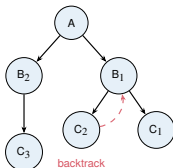
Mechanism	Closed by	Signal	Characteristic failure
Self-Refine [16]	S	Model critiques own output	Plateaus; nothing can contradict the generator
Self-debug, no tests [3]	S	Self-generated explanation	Explains the bug as intended behaviour
Self-debug, with tests [3]	O+S	Test verdict + model diagnosis	Bounded by test coverage
Reflexion [22]	O or P	Task reward where one exists; on HumanEval, self-generated tests	Filed as external, but its code result rests on a proxy
SelfEvolve [12]	O	Interpreter errors, traces	Silent (non-crashing) semantic bugs
D4C [28]	O	Failing tests + full function	Plausible-but-incorrect patches
CRITIC [10]	O	Self-critique routed <i>through</i> an external tool	Needs a tool covering the error class
Agentless validate [27]	O+P	Regression tests + generated reproduction tests	The proxy half inherits the model's misreading
Best-of- N + critic [21]	O+P	Executed rollouts ranked by a trained critic	Critic is a distilled proxy; cost scales with N

Backup: how plans are shaped

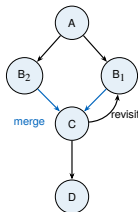
Chain of Thought



Tree of Thoughts



Graph of Thoughts



The evaluator, not the branching, makes the structure useful — and it is what a repository-scale agent finds hardest to supply. *What is a half-finished patch worth, before any test has run?* SWE-Search [1] builds one (MCTS + LLM value function, +23% relative) — and must invoke a model at every node.

Backup: context and cost

- **Longer windows do not dissolve the bottleneck.** Retaining full *dependency* context works best; padding with loosely related code *degrades* correctness [18].
- **Context explosion and semantic drift.** Appending every observation buries the critical early information [15].
- **Cost is quadratic in turns.** The whole prefix is re-sent each turn, so tokens grow as $O(n^2)$ [9]. A property of the protocol, not an empirical finding; prompt caching cuts the constant, not the growth rate.
- **Resolve rate decouples from compute** [7]. Agentless leads on resolve rate (48 % with Qwen3-32B) at 83.1 API calls and 209.4 s per issue. The frontier is accuracy *per dollar*.
- **And verification is the human bottleneck.** In an RCT, experienced developers were 19 % *slower* with early-2025 AI tools — while believing they had been 20 % faster [2].

References I

- [1] A. Antoniadou, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang.
SWE-search: Enhancing software agents with monte carlo tree search and iterative refinement.
arXiv preprint arXiv:2410.20285, 2024.
ICLR 2025.
- [2] J. Becker, N. Rush, E. Barnes, and D. Rein.
Measuring the impact of early-2025 AI on experienced open-source developer productivity.
arXiv preprint arXiv:2507.09089, 2025.
METR randomised controlled trial.
- [3] X. Chen, M. Lin, N. Schärli, and D. Zhou.
Teaching large language models to self-debug.
In The Twelfth International Conference on Learning Representations, 2024.
- [4] Cognition AI.
Introducing devin, the first AI software engineer.
<https://cognition.ai/blog/introducing-devin>, 2024.
Accessed: 2026-07-09.
- [5] Cursor.
Reward hacking is swamping model intelligence gains.
<https://cursor.com/blog/reward-hacking-coding-benchmarks>, 2026.
Accessed: 2026-06-30.
- [6] X. Deng et al.
SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?
arXiv preprint arXiv:2509.16941, 2025.
- [7] Z. Fan, K. Vasilevski, D. Lin, B. Chen, Y. Chen, Z. Zhong, J. M. Zhang, P. He, and A. E. Hassan.

References II

SWE-effi: Re-evaluating software AI agent system effectiveness under resource constraints.

arXiv preprint arXiv:2509.09853, 2025.

[8] J. Ganhotra.

Cracking the code: How difficult are SWE-bench-verified tasks really?

<https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html>, 2025.

Blog post, accessed: 2025-06-01.

[9] P. Gao and C. Peng.

More with less: An empirical study of turn-control strategies for efficient coding agents.

arXiv preprint arXiv:2510.16786, 2025.

[10] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen.

CRITIC: Large language models can self-correct with tool-interactive critiquing.

In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

arXiv:2305.11738.

[11] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou.

Large language models cannot self-correct reasoning yet.

In *The Twelfth International Conference on Learning Representations*, 2024.

[12] S. Jiang, Y. Wang, and Y. Wang.

SelfEvolve: A code evolution framework via large language models.

arXiv preprint arXiv:2306.02907, 2023.

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan.

SWE-bench: Can language models resolve real-world GitHub issues?

arXiv preprint arXiv:2310.06770, 2024.

References III

ICLR 2024.

- [14] S. K. Lahiri, S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, M. Musuvathi, P. Choudhury, C. von Veh, J. P. Inala, C. Wang, and J. Gao. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022. TiCoder.
- [15] S. Liu, J. Yang, B. Jiang, Y. Li, J. Guo, X. Liu, and B. Dai. Context as a tool: Context management for long-horizon SWE-agents. *arXiv preprint arXiv:2512.22087*, 2025.
- [16] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [17] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 1:2332–2354, 2024. *arXiv:2310.10996*.
- [18] N. L. H. Nam et al. On the impacts of contexts on repository-level code generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.
- [19] OpenAI. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Accessed: 2025-06-01.

References IV

- [20] OpenAI.
Why SWE-bench verified no longer measures frontier coding capabilities.
<https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>, 2026.
Accessed: 2026-06-30.
- [21] OpenHands.
SOTA on SWE-Bench verified with inference-time scaling and critic model.
<https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and-critic-model>, 2025.
Accessed: 2026-05-28.
- [22] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao.
Reflexion: Language agents with verbal reinforcement learning.
In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.
- [23] M. Suri, X. Li, M. Shojai, S. Han, C.-C. Hsu, S. Garg, A. Deshmukh, and V. Kumar.
CodeScout: Contextual problem statement enhancement for software agents.
2026.
arXiv:2603.05744v2, 9 April 2026. ScaffoldXmodel grid is the paper's no-augmentation baseline (its Figure 3), not its contribution.
- [24] SWE-agent Team.
mini-SWE-agent.
<https://github.com/SWE-agent/mini-swe-agent>, 2026.
~100 lines of Python for the agent class; reports >74% on SWE-bench Verified. Accessed: 2026-07-12.
- [25] S. Vijayvargiya, X. Zhou, A. Yerukola, M. Sap, and G. Neubig.
Interactive agents to overcome ambiguity in software engineering.
In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026.
arXiv:2502.13069; later revised as "Ambig-SWE".

References V

- [26] X. Wang, B. Li, Y. Song, F. F. Xu, G. Neubig, et al.
OpenHands: An open platform for AI software developers as generalist agents.
In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
[arXiv:2407.16741](#).
- [27] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang.
AGENTLESS: Demystifying LLM-based software engineering agents.
arXiv preprint arXiv:2407.01489, 2024.
- [28] J. Xu, Y. Fu, S. H. Tan, and P. He.
Aligning the objective of LLM-based program repair.
In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025.
D4C. [arXiv:2404.08877](#).
- [29] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press.
SWE-agent: Agent-computer interfaces enable automated software engineering.
arXiv preprint arXiv:2405.15793, 2024.
- [30] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao.
ReAct: Synergizing reasoning and acting in language models.
arXiv preprint arXiv:2210.03629, 2023.
ICLR 2023.
- [31] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury.
AutoCodeRover: Autonomous program improvement.
In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.